CYBERSECURITY MINI PROJECT

PASSWORD STRENGTH CHECKER

A Python-Based Password Evaluation Tool

Project Title	Password Strength Checker
Domain	Cybersecurity
Programming Language	Python 3
Prepared by	_____
Register Number	_____
Department / Class	_____
College	_____
Academic Year	_____

Submitted in partial fulfilment of the requirements for the relevant course. Fill in the student and institution details before submission.

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Existing System and Proposed System
6. Requirements Specification
7. System Design and Methodology
8. Algorithm and Flow
9. Implementation
10. Testing and Sample Results
11. Advantages and Limitations
12. Future Enhancements
13. Security and Privacy Considerations
14. Conclusion
15. References

1. Abstract

The Password Strength Checker is a beginner-friendly cybersecurity application developed using Python. It evaluates a password against five basic checks: minimum length, uppercase letters, lowercase letters, numbers, and special characters. The program assigns a score out of five, classifies the result as Weak, Medium, or Strong, and displays suggestions for missing checks. The project demonstrates basic secure-password awareness and the use of conditional statements, functions, lists, and regular expressions in Python.

2. Introduction

Passwords are commonly used to protect online accounts, applications, and digital information. Passwords that are short, predictable, or reused can be easier to guess. A password checker can help users understand some basic characteristics that contribute to password quality.

This project is implemented as a command-line Python program. It does not require a database, internet connection, or third-party package. The password is evaluated when entered, and the program displays the result and improvement suggestions.

3. Problem Statement

Many users create passwords without checking their length or character variety. The project addresses this awareness problem by providing a simple tool that evaluates five basic rules and gives immediate feedback.

4. Objectives

• Evaluate a password using five basic character and length checks.

• Display a score from 0 to 5.

• Classify the result as Weak, Medium, or Strong.

• Suggest improvements when one or more checks are missing.

• Promote awareness of safer password creation.

5. Existing System and Proposed System

5.1 Existing System

Without a checker, users may rely only on their own judgement when deciding whether a password is strong. They may overlook length, character variety, or common password choices.

5.2 Proposed System

The proposed system checks five defined rules, calculates a score, assigns a strength category, and prints suggestions. It is lightweight and can run locally with Python 3.

5.3 Scope

The current version is an educational prototype. It does not check password leaks, estimate guessing time, or guarantee security. Its output should be treated as basic guidance rather than a definitive security rating.

6. Requirements Specification

6.1 Hardware Requirements

Component	Requirement
Computer	Desktop or laptop
Memory	2 GB RAM or more recommended
Storage	At least 100 MB free space
Input/output	Keyboard and display

6.2 Software Requirements

Software	Requirement
Operating system	Windows or Linux; macOS is also suitable
Language	Python 3
Editor	IDLE, VS Code, or PyCharm
Modules	re and getpass (built into Python)

6.3 Functional Requirements

• Accept a password from the user.

• Check length and four character categories.

• Calculate a score out of five.

• Display the strength category.

• Display suggestions for failed checks.

6.4 Non-Functional Requirements

• Usability: results and suggestions should be easy to understand.

• Performance: evaluation should complete quickly for normal input.

• Privacy: the application should not save or transmit the password.

• Portability: it should run on systems that support Python 3.

7. System Design and Methodology

7.1 Input

The program accepts a password through a terminal prompt. The getpass module hides typed input in supported terminals.

7.2 Processing

The program uses regular expressions to check for uppercase letters, lowercase letters, digits, and special characters. It also checks whether the password contains at least eight characters. Each passed check adds one point.

7.3 Output

The program displays the strength category, score out of five, and suggestions for any missing checks.

7.4 Score Categories

Score	Category
0–2	Weak
3–4	Medium
5	Strong

8. Algorithm and Flow

1. Start the program.
2. Read a password from the user.
3. Initialize score to zero and create an empty suggestions list.
4. Check the minimum length of eight characters.
5. Check uppercase, lowercase, numeric, and special characters.
6. Increase the score for each passed check and add suggestions for failed checks.
7. Assign Weak for scores 0–2, Medium for scores 3–4, or Strong for score 5.
8. Display the category, score, and suggestions.
9. Stop the program.

9. Implementation

The program is written in Python and uses a function named check_password(password). The re module supports pattern matching, while getpass hides password input in supported terminals. No third-party libraries are needed.

9.1 Source Code

```
import re
import getpass


def check_password(password):
    score = 0
    suggestions = []

    if len(password) >= 8:
        score += 1
    else:
        suggestions.append("Use at least 8 characters.")

    if re.search(r"[A-Z]", password):
        score += 1
    else:
        suggestions.append("Add an uppercase letter.")

    if re.search(r"[a-z]", password):
        score += 1
    else:
        suggestions.append("Add a lowercase letter.")

    if re.search(r"[0-9]", password):
        score += 1
    else:
        suggestions.append("Add a number.")

    if re.search(r"[^A-Za-z0-9]", password):
        score += 1
    else:
        suggestions.append("Add a special character.")

    if score <= 2:
        strength = "WEAK"
    elif score <= 4:
        strength = "MEDIUM"
```

```
    else:
        strength = "STRONG"

    print("\\nPassword Strength:", strength)
    print("Score:", score, "/ 5")

    if suggestions:
        print("\\nSuggestions:")
        for item in suggestions:
            print("-", item)
    else:
        print("Good! Your password meets all five checks.")


print("PASSWORD STRENGTH CHECKER")
password = getpass.getpass("Enter a password: ")
check_password(password)
```

9.2 How to Run

1. Install Python 3 if it is not already installed.

2. Save the code as password_checker.py.

3. Open a terminal in the file's folder.

4. Run python password_checker.py (or python3 password_checker.py on some systems).

5. Enter a made-up test password and review the result.

10. Testing and Sample Results

The following are illustrative test cases. Actual output depends on the password entered. Use made-up examples for demonstrations rather than real account passwords.

Test	Example input characteristics	Expected category
1	Short; lowercase and digits only	Weak
2	At least 8 characters; four of five checks passed	Medium
3	At least 8 characters and all four character categories	Strong

10.1 Sample Output

```
PASSWORD STRENGTH CHECKER
Enter a password: [input hidden]

Password Strength: STRONG
Score: 5 / 5
Good! Your password meets all five checks.
```

11. Advantages and Limitations

11.1 Advantages

• Simple and suitable for beginners.

• Uses built-in Python modules.

• Provides immediate feedback.

• Can run locally without an internet connection.

11.2 Limitations

• Checks only five basic rules.

• Does not detect all common or compromised passwords.

• Does not estimate how quickly a password could be guessed.

• A Strong result does not guarantee a secure password.

12. Future Enhancements

• Add a graphical user interface using Tkinter.

• Develop a web version using Flask or Django.

• Include a password generator.

• Add a visual strength meter.

• Use a privacy-preserving method to check known compromised passwords.

13. Security and Privacy Considerations

The application should not log, store, or transmit the password. Demonstrations should use made-up passwords. For real-world systems, password security should also consider length, uniqueness, blocklists for common passwords, secure password storage by services, and resistance to guessing. Character variety alone is not a complete measure.

14. Conclusion

The Password Strength Checker is a simple cybersecurity mini project implemented using Python. It checks a password's length and character categories, calculates a score, classifies the result, and provides suggestions. The project helps beginners learn basic password-security concepts and Python programming. It is intended for education and should not be treated as a complete security assessment.

15. References

• Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/

• Python Software Foundation, re — Regular expression operations: https://docs.python.org/3/library/re.html

• Python Software Foundation, getpass — Portable password input: https://docs.python.org/3/library/getpass.html